

PLAN: Parallel Liquid-Inspired Approximation Network for Efficient Representation Learning in Flexible Job Shop Scheduling

Dhivya Dharshini Kannan¹, Wei Zhang^{1*}, Jieyi Bi², Yingpeng Du², Tianjun Wei², Jie Zhang²,
Zuming Liu³, Anupam Trivedi⁴

¹Singapore Institute of Technology (SIT)

²Nanyang Technological University (NTU)

³Shanghai Jiao Tong University

⁴Agency for Science, Technology and Research (A*STAR)

Abstract

Deep reinforcement learning (DRL) approaches for flexible job shop scheduling (FJSP) heavily rely on attention-centric architectures to achieve state-of-the-art performance. However, these models suffer from excessive parameter counts and prohibitive inference latency as problem scales expand. While liquid neural networks (LNNs) offer a parameter-efficient alternative for modeling adaptive state evolution, their inherently sequential dynamics bottleneck computational efficiency. To resolve this trade-off, we propose **PLAN** (Parallel Liquid-inspired Approximation Network), a lightweight representation learning framework that reformulates continuous liquid-state dynamics into a discretized and parallelizable formulation. PLAN structurally decouples state evolution from context aggregation, where liquid-inspired updates handle the primary evolving state representation, and a lightweight context aggregation module provides complementary global context. Furthermore, PLAN acts as a versatile, plug-and-play backbone that generalizes to complex FJSP variants, pairing with a compact stochastic module for stochastic FJSP and replacing heavy heterogeneous graph transformers in multi-faceted dynamic FJSP. Extensive evaluations across deterministic, stochastic, and multi-faceted dynamic FJSP benchmarks show that PLAN reduces the average makespan by 1.2%, 1.4%, and 2.3%, respectively, compared with the corresponding state-of-the-art baselines, with the improvement reaching 10.2% in one benchmark setting. PLAN also reduces average inference latency by 13.2%, 31.7%, and 26.9%, respectively, with a maximum reduction of 69.2% on the largest instances, while using only 22–47% of the baseline parameters.

Introduction

Job scheduling is a fundamental combinatorial optimization problem with broad applications across industrial, computing, and service systems (Kwan et al. 2026). Flexible Job Shop Scheduling (FJSP), one of its most widely studied formulations, has been applied to areas such as automotive assembly (Kim et al. 2022), healthcare scheduling (Burdett and Kozan 2018), and semiconductor fabrication (Ghaedy-Heidary et al. 2024). In FJSP, each operation of a job can be assigned to one of multiple eligible machines while satisfying precedence and resource constraints (Wang et al. 2026). Due to its NP-hard nature (Xie et al. 2019), obtaining high-quality

schedules within acceptable computation time becomes increasingly difficult as the problem size grows. Classical methods, including tabu search (Brandimarte 1993), genetic algorithms (Li et al. 2019), and dispatching heuristics (Li and Gao 2016), often struggle to achieve a good trade-off between solution quality and computational efficiency, especially in large-scale or dynamic scheduling environments.

Recent progress in deep reinforcement learning (DRL) has demonstrated the potential of learning-based schedulers to generate high-quality schedules with fast inference after training (Kaleta and Śliwiński 2026; Liu et al. 2025). Existing state-of-the-art (SOTA) DRL-based approaches, such as HGNN (Song et al. 2023) and DANIEL (Wang et al. 2024), achieve strong performance through deep attention-based representation learning. However, these architectures rely on multiple full attention blocks and scheduling-specific auxiliary components to model interactions among operations and machines, increasing parameter count, memory footprint, and inference latency. As the numbers of operations and machines grow, these components must capture increasingly complex interactions, resulting in higher computational overhead for large scheduling instances.

Scheduling decisions are made sequentially, and each decision immediately changes the scheduling state, including machine availability and operation readiness. The scheduling state therefore evolves throughout the decision process. Deep attention-based architectures are effective at modelling interactions among scheduling entities (Song et al. 2023; Wang et al. 2024). However, they are not explicitly designed to model the decision-dependent evolution of scheduling states throughout the sequential decision process. This motivates the exploration of alternative representation learners that can propagate scheduling information efficiently while remaining compact. Liquid neural networks (LNNs), originally designed for continuous-time dynamic systems, support adaptive state updates with few parameters (Akpınar et al. 2025), making them a promising mechanism for representing evolving states. However, their inherently sequential state evolution limits computational efficiency and prevents the state updates from being efficiently processed in parallel.

Motivated by these observations, we propose PLAN, a Parallel Liquid-Inspired Approximation Network, as a lightweight representation learning framework for DRL-based FJSP. PLAN reformulates the sequential liquid-state

*Corresponding author.

evolution of LNNs into a discretized and parallelizable representation learning process. Its liquid-inspired state updates perform the primary representation learning, while a shallow attention module provides complementary global context. This design shifts the main representation learning from deep attention to liquid-inspired state updates, substantially reducing architectural complexity. Experiments under deterministic, stochastic, and multi-faceted dynamic FJSP settings show that PLAN reduces model complexity and inference latency while improving scheduling performance.

The main contributions are summarized as follows.

- We propose PLAN, a lightweight representation learner combining liquid-inspired state updates with a shallow attention module for efficient FJSP scheduling.
- We develop a parallelizable liquid-inspired representation learner by reformulating the sequential ordinary differential equation (ODE) dynamics of LNNs through an Euler-based approximation, preserving adaptive state updates while enabling parallel computation.
- We extend PLAN to stochastic FJSP using a smaller stochastic processing module (SPM) and to multi-faceted dynamic FJSP by replacing the original heterogeneous graph transformer (HGT). Experiments across deterministic and dynamic settings show reduced model size and inference latency together with improved performance.

Problem Formulation and Scheduling Settings

FJSP includes sets of jobs $J = J_1, J_2, \dots, J_n$ and machines $M = M_1, M_2, \dots, M_m$, where each job J_i consists of an ordered sequence of operations $O_i = O_{i1}, O_{i2}, \dots, O_{in_i}$, and n_i denotes the number of operations in J_i . The full set of operations is denoted as $O = \bigcup_i O_i$. An operation O_{ij} is assigned to one machine from its compatible machine set $M_{ij} \subseteq M$. When O_{ij} is processed on machine $M_k \in M_{ij}$, it requires processing time $p_{ij}^k > 0$, and C_{ij} denotes its completion time. The objective is to minimize the makespan $C_{\max}$, i.e., the completion time of the last completed operation.

$$C_{\max} = \max_{O_{ij} \in O} C_{ij}, \quad (1)$$

A feasible schedule must satisfy the precedence constraints within each job, assign exactly one compatible machine to each operation, and ensure that each machine processes at most one operation at a time.

Processing times may be uncertain due to factors such as resource conditions, execution delays, and unexpected disturbances, and their exact values may be unavailable before scheduling. To model this uncertainty, we consider stochastic FJSP with stochastic processing times (Smit et al. 2025), where the deterministic processing time p_{ij}^k is replaced by a random variable P_{ij}^k , making the operation completion times and final makespan random variables. We further evaluate PLAN under the multi-faceted dynamic FJSP setting, following the benchmark configuration and dynamic-event protocol established in (Liu et al. 2026).

Methodology

This section presents the DRL formulation, the PLAN framework and its key components and the learning procedure.

MDP Formulation

The scheduling problem is formulated as a Markov decision process (MDP), where operation-machine assignment decisions are made sequentially until all operations have been assigned to machines. At each decision step, a DRL agent selects an operation-machine pair based on the current scheduling state and receives a reward reflecting the quality of the resulting schedule (Wang et al. 2024). The MDP is defined by the state space $\mathcal{S}$, action space $\mathcal{A}$, transition function $\mathcal{P}$, and reward function $\mathcal{R}$, which are described below.

State. The state s_t denotes the current scheduling status at decision step t . For deterministic FJSP, it consists of three categories of entity features, given by $s_t = \{H_O, H_M, H_{OM}\}$, where H_O , H_M , and H_{OM} denote the operation, machine, and operation-machine pair features, respectively. The operation features describe the processing and scheduling status of operations, the machine features characterize machine utilization and availability, and the pair features capture the compatibility and processing relationships between candidate operations and machines. For stochastic FJSP, the state additionally includes sampled processing-time scenarios representing multiple possible realizations of processing-time uncertainty. For the multi-faceted dynamic setting, it further captures changes in the scheduling environment, such as machine breakdowns and new job arrivals.

Action. At each decision step t , the agent selects a feasible action $a_t = (O_{ij}, M_k)$ that assigns operation O_{ij} to machine M_k . The action space $\mathcal{A}_t$ contains all feasible operation-machine pairs at decision step t satisfying operation precedence and machine compatibility constraints. The same action definition is used across the deterministic, stochastic, and multi-faceted dynamic settings.

State Transition. Once an action a_t is executed, the scheduling environment updates the operation status, machine availability, and feasible action space according to the operation precedence and machine constraints, producing the next state s_{t+1} from the current state s_t .

Reward. The reward function is designed to encourage schedules with smaller makespan. At state s_t , the estimated makespan is denoted by $\hat{C}_{\max}(s_t)$. After an action, the immediate reward is formulated as the difference between the estimated makespan of the current and next states, $r_t = \hat{C}_{\max}(s_t) - \hat{C}_{\max}(s_{t+1})$. A positive reward indicates an improvement, while a negative reward indicates a reduction in scheduling quality. For stochastic FJSP, the estimated makespan is evaluated over n sampled processing-time scenarios, denoted by $\{\hat{C}_{\max}^1(s_t), \hat{C}_{\max}^2(s_t), \dots, \hat{C}_{\max}^n(s_t)\}$. We adopt Value-at-Risk (VaR) as the risk-sensitive scheduling objective, i.e., $f(s_t) = \text{VaR}_\alpha(\hat{C}_{\max}(s_t))$, and define the immediate reward as $r_t = f(s_t) - f(s_{t+1})$.

Policy. The policy $\pi_\theta(a_t|s_t)$ maps the current scheduling state to a probability distribution over feasible actions. The policy parameters θ are learned through interactions with the scheduling environment.

PLAN Framework

Figure 1 illustrates the overall architecture of PLAN. First, liquid-inspired state dynamics enable adaptive representation

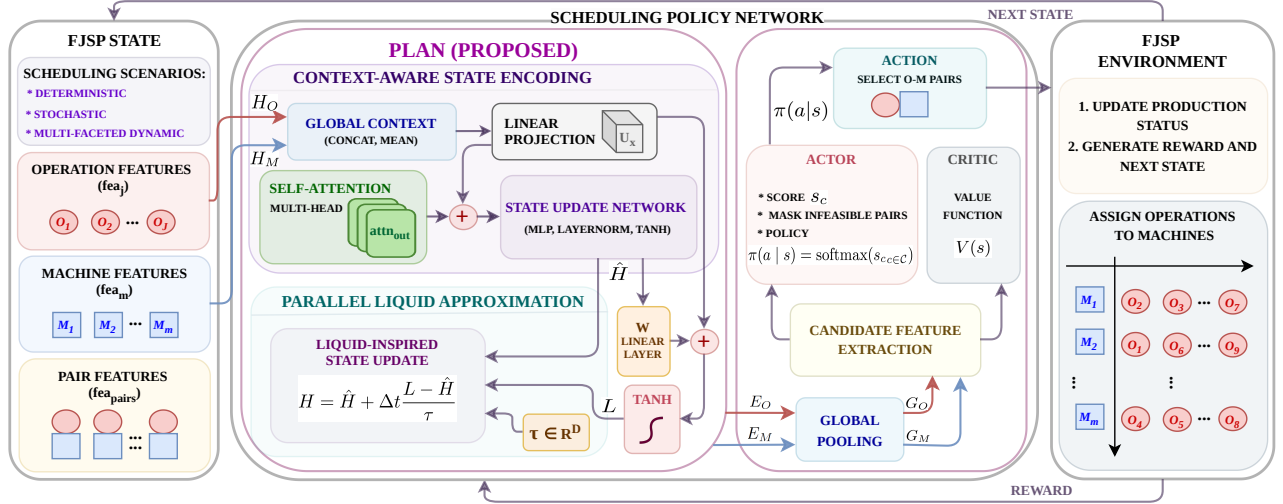


Figure 1: Overview of the proposed PLAN framework for FJSP.

learning for the evolving scheduling environment. Second, parallel approximation enables efficient liquid state updates without sequential ODE integration, making the framework suitable for large-scale scheduling. To realize these ideas, PLAN aggregates global contextual information, and performs parallel liquid state updates to learn operation and machine representations for downstream scheduling decisions. Each component is described in detail below.

Liquid-Inspired State Dynamics FJSP is a dynamic decision-making problem in which machine workloads, operation statuses, and feasible actions continuously evolve throughout the scheduling process. Therefore, the representation learning model should capture not only the relationships among scheduling entities but also the evolution of the scheduling state. LNNs (Kannan et al. 2026) naturally model such evolving states through adaptive state dynamics. Instead of learning a static mapping, the hidden state evolves continuously according to the current hidden state and scheduling input, allowing the representation to adapt as the scheduling environment changes. This formulation motivates PLAN, which develops an efficient parallel approximation for scheduling. For input $x(t)$ and hidden state $h(t)$, the continuous liquid dynamics are formulated as,

$$\frac{dh_t}{dt} = -\frac{h_t}{\tau} + \sigma(W_h h_t + W_x x_t), \quad (2)$$

where τ is the learnable time constant, W_h and W_x are trainable weight matrices, and $\sigma(\cdot)$ denotes a nonlinear activation function. Given an initial hidden state h_0 , the ODE in Eq. (2) is integrated over the time interval $[0, T]$ to obtain the evolved hidden state $h(T)$, where T denotes the integration horizon,

$$h(T) = \text{ODE}\left(\frac{dh}{dt}, h_0\right). \quad (3)$$

Direct numerical integration introduces additional computational overhead and is not well suited to FJSP, where decisions are made at discrete scheduling steps. We therefore

adopt a first-order Euler discretization with time step Δt ,

$$h_{t+1} = h_t + \Delta t \frac{dh_t}{dt}. \quad (4)$$

Substituting Eq. (2) into Eq. (4) gives,

$$h_{t+1} = h_t + \Delta t \left(-\frac{h_t}{\tau} + \sigma(W_h h_t + W_x x_t) \right). \quad (5)$$

Although Eq. (5) converts the continuous dynamics into a discrete formulation, the hidden state is still updated sequentially because each state depends on the previously computed state. This sequential dependency limits parallel execution and reduces computational efficiency on modern hardware. To address this limitation, PLAN reformulates the liquid state update as a parallel approximation, as described below, while preserving the adaptive characteristics of liquid dynamics.

Context-Aware Representation Learning The input state consists of heterogeneous operation and machine features. To enable unified representation learning, PLAN first projects each feature set X into a common latent space through a trainable linear transformation $\mathcal{U}(\cdot)$, yielding $U_X = \mathcal{U}(X)$. Although the projected features preserve the local information of individual scheduling entities, they do not explicitly capture the contextual relationships among them. Therefore, PLAN employs a lightweight multi-head attention (MHA) module to aggregate global scheduling context. Unlike current SOTA schedulers that rely on deep attention blocks (Islam et al. 2024) as the main representation learner, PLAN uses attention only to aggregate scheduling context, while the liquid-inspired state update performs the main representation learning. Therefore, PLAN employs only a lightweight MHA module, without the stacked feed-forward, normalization, and residual blocks of a full Transformer encoder. The contextual representation is computed as $A = \text{MHA}(U_X, U_X, U_X)$, where the projected features serve as the query, key, and value to perform self-attention

within the same feature set, enabling parallel information exchange and aggregation of global scheduling context. The projected features and contextual representation are then fused to estimate the initial hidden state,

$$Z = [U_X; A], \quad \hat{H} = \phi(Z), \quad (6)$$

where $[\cdot; \cdot]$ denotes feature concatenation, and $\phi(\cdot)$ is the state estimation network consisting of two fully connected layers with layer normalization and a nonlinear activation. It maps the fused local and global contextual features to the initial hidden state for the subsequent liquid approximation.

Parallel Liquid Approximation The estimated hidden states initialize the liquid dynamics. However, directly applying the Euler update in Eq. (5) still requires recurrent state propagation, preventing all hidden states from being computed simultaneously. PLAN therefore approximates the liquid state evolution through a parallel formulation that preserves the adaptive characteristics of liquid dynamics while eliminating sequential dependencies. The liquid-inspired nonlinear response is first computed from the estimated hidden state as,

$$L = \tanh(W\hat{H} + U_X), \quad (7)$$

where W is a trainable weight matrix, and U_X provides the projected scheduling features as the external input. The tanh activation preserves the bounded nonlinear state transition of the original liquid dynamics while operating on the estimated hidden state. The liquid state is then refined using a single liquid-inspired correction step,

$$H = \hat{H} + \Delta t \frac{L - \hat{H}}{\tau}, \quad (8)$$

which can be viewed as a parallel approximation of a single Euler-style liquid update without recurrent state propagation. Consequently, all hidden states can be refined simultaneously through batched matrix operations. The resulting representations capture both the contextual relationships among scheduling entities and the nonlinear state adaptation inherited from the liquid formulation, providing informative embeddings for downstream scheduling decisions.

Operation and Machine Encoding PLAN processes the operation and machine features using two independent encoders with the same architecture. Before encoding, PLAN summarizes each feature set through mean pooling to capture its global scheduling context and concatenates this context with every corresponding entity feature, yielding $H_O^c = [H_O; \text{mean}(H_O)]$ and $H_M^c = [H_M; \text{mean}(H_M)]$, where $\text{mean}(\cdot)$ computes the average feature vector across all entities and $[\cdot; \cdot]$ denotes feature concatenation. The resulting context-enhanced features are then independently processed through the context-aware representation learning and parallel liquid approximation introduced above, producing the encoded representations E_O and E_M , respectively. Finally, the encoded representations are pooled as $G_O = \text{pool}(E_O)$ and $G_M = \text{pool}(E_M)$ to obtain global operation and machine embeddings for the subsequent decision network.

Algorithm 1: PLAN Encoding and Decision Procedure

```

1: Input: Operation features  $H_O$ , machine features  $H_M$ , candidate operations  $\mathcal{C}$ , and pair features  $H_{OM}$ 
2: Encode operation context:  $H_O^c = [H_O; \text{mean}(H_O)]$ 
3: Encode machine context:  $H_M^c = [H_M; \text{mean}(H_M)]$ 
4: for  $X \in H_O, H_M$  do
5:   Project features:  $U_X = \mathcal{U}(X)$ 
6:   Aggregate scheduling context:  $A = \text{MHA}(U_X, U_X, U_X)$ 
7:   Fuse local and contextual features:  $Z = [U_X; A]$ 
8:   Estimate hidden state:  $\hat{H} = \phi(Z)$ 
9:   Compute liquid response:  $L = \tanh(W\hat{H} + U_X)$ 
10:  Apply liquid approximation:  $H = \hat{H} + \Delta t(L - \hat{H})/\tau$ 
11: end for
12: Obtain embeddings:  $G_O = \text{pool}(H_O), G_M = \text{pool}(H_M)$ 
13: for  $c \in \mathcal{C}$  do
14:   Fuse representation:  $F_c = [H_O^c; H_M^c; G_O; G_M; H_{OM}^c]$ 
15:   Compute candidate score:  $s_c = \text{Actor}(F_c)$ 
16: end for
17: Mask infeasible operation-machine pairs
18: Compute scheduling policy:  $\pi(a | s) = \text{softmax}(s_{cc \in \mathcal{C}})$ 
19: Estimate state value:  $V(s) = \text{Critic}([G_O; G_M])$ 
20: Output: Policy distribution  $\pi(a | s)$  and state value  $V(s)$ 

```

SPM-PLAN

For the stochastic FJSP, PLAN is integrated with SPM (Smit et al. 2025), which summarizes sampled processing-time scenarios into a compact stochastic representation. For a set of n scenario embeddings $H = h_1, h_2, \dots, h_n$, SPM avoids applying full self-attention across all scenarios. Instead, it employs a small set of inducing vectors I to approximate their global interactions through two cross-attention blocks (CABs). The resulting stochastic representation is then obtained through mean pooling and formulated as,

$$\text{SPM}(H) = \text{mean}(\text{CAB}(H, \text{CAB}(I, H))). \quad (9)$$

The stochastic representation is concatenated with the deterministic representation as $h = [h^{\text{det}}; \text{SPM}(H)]$, enriching the scheduling state with processing-time uncertainty. Since PLAN already performs contextual representation learning and adaptive state refinement, a compact SPM is sufficient to extract stochastic information without introducing unnecessary computational overhead. Consequently, SPM-PLAN preserves effective uncertainty modeling while remaining lightweight and enabling faster inference.

Training via PPO

We train PLAN using proximal policy optimization (PPO). The actor network parameterized by θ produces the previously defined policy $\pi_\theta(a_t | s_t)$ over feasible actions. During training, actions are sampled from this distribution to encourage exploration, while interactions with the environment generate rewards and subsequent states to form scheduling trajectories. The critic network estimates the state value $V_\phi(s_t)$, which is used to compute advantage estimates that quantify the relative quality of sampled actions. PPO optimizes the actor through a clipped surrogate objective that limits excessive policy changes and stabilizes training. Through repeated interactions with the environment, the policy progressively learns to minimize the FJSP makespan.

Table 1: Performance comparison on the small deterministic FJSP benchmarks SD1 and SD2. Gap (%) denotes the relative makespan difference from the OR-Tools reference, and the average schedule generation time is reported. Lower values are better.

Data	Size	OR-Tools	Greedy						Sampling					
			DANIEL			PLAN			DANIEL			PLAN		
		Makespan	Makespan	Gap	Time	Makespan	Gap	Time	Makespan	Gap	Time	Makespan	Gap	Time
SD1	10×5	96.32	107.97	12.10	0.48	107.36	11.47	0.33	102.36	6.27	1.15	101.48	5.36	0.81
	20×5	188.15	197.66	5.06	0.90	197.78	5.12	0.65	193.83	3.02	2.41	193.14	2.65	1.75
	15×10	143.53	160.78	12.02	1.23	159.21	10.93	0.96	152.51	6.26	3.83	150.68	4.98	3.04
	20×10	195.98	199.14	1.61	1.62	198.21	1.14	1.29	195.30	−0.35	5.31	193.38	−1.33	4.30
SD2	10×5	326.24	413.91	26.87	0.40	407.86	25.02	0.31	366.59	12.37	1.11	362.46	11.10	0.81
	20×5	602.04	673.28	11.83	1.03	661.45	9.87	0.83	632.85	5.12	3.09	624.68	3.76	2.68
	15×10	377.17	588.14	55.94	1.53	587.84	55.86	1.22	519.78	37.81	4.93	515.61	36.70	4.36
	20×10	464.16	606.14	30.59	2.01	603.43	30.00	1.65	552.09	18.94	6.09	550.11	18.52	5.98

Experiments

This section evaluates PLAN under deterministic, stochastic, and multi-faceted dynamic FJSP settings.

Datasets and Configuration

We evaluate PLAN on deterministic, stochastic, and multi-faceted dynamic FJSP benchmarks. The deterministic evaluation uses the synthetic SD1 and SD2 datasets (Wang et al. 2024), covering small (10×5 , 20×5 , 15×10 , 20×10), medium (30×10 , 40×10), and large (100×10 , 200×5) problem scales, together with the public Brandimarte (Brandimarte 1993) and Hurink (Li et al. 2019) benchmarks. For stochastic scheduling, we adopt the SD3 benchmark (Smit et al. 2025), where processing times are generated by sampling around the median deterministic processing times. For multi-faceted dynamic scheduling, we use the benchmark and DRL framework from (Liu et al. 2026). Each benchmark contains 100 instances for every problem scale. We evaluate both greedy and sampling action-selection strategies, where greedy selects the action with the highest policy probability and sampling draws actions from the policy distribution. All experiments are repeated with five random seeds (0–4), and the reported results are averaged across runs.

For deterministic FJSP, we adopt the implementation settings of DANIEL, the current SOTA, to ensure a fair comparison and compare PLAN against it. We additionally report OR-Tools as a reference solver. Since exact optimization becomes computationally expensive for large-scale FJSP with complex constraints, following standard practice, OR-Tools is executed with a 30-minute time limit for each instance, whereas DRL methods, e.g., PLAN, generate schedules almost instantly after training. For stochastic FJSP, we evaluate SPM-PLAN by integrating PLAN with SPM and compare it with SPM-DAN, the stochastic extension of DANIEL. For multi-faceted dynamic FJSP, we follow the implementation settings and evaluation protocol of HGT, the SOTA method for this setting. All models are implemented in PyTorch and trained on a workstation equipped with an NVIDIA RTX PRO 5000 Blackwell GPU with 48 GB of memory.

Deterministic FJSP

We compare PLAN with DANIEL and OR-Tools on the small deterministic benchmarks from SD1 and SD2, where training

and testing use the same instance sizes. As shown in Table 1, PLAN achieves a smaller makespan gap than DANIEL in seven of the eight settings across greedy and sampling decoding, while also requiring less inference time.

To evaluate cross-scale generalization, we train PLAN and DANIEL only on 10×5 instances and test them on unseen medium- and large-scale instances and public benchmarks. As shown in Table 2, PLAN outperforms DANIEL in all but two settings and even surpasses OR-Tools in several cases. Its advantage is maintained across substantially larger and more diverse problem settings, indicating that PLAN generalizes effectively beyond the smallest training scale.

Supplementary Table S2 further shows that PLAN consistently outperforms representative priority dispatching rules (PDRs) (Sels, Gheysen, and Vanhoucke 2012), including shortest processing time (SPT) and most work remaining (MWKR), as well as the DRL-based HGNN across diverse benchmark settings. Moreover, PLAN reduces the parameter count by 53.16% (28,834 to 13,560) and halves the model size from 136kB to 68kB while achieving better scheduling performance, confirming that its accuracy gains do not come at the cost of model complexity or deployment efficiency.

Stochastic FJSP

We show the stochastic scheduling results in Table 3, where 32 and 8 denote the SPM hidden dimensions used in the two SPM-PLAN variants, respectively. Similar to the deterministic FJSP experiments, for medium and large datasets, we evaluate generalization using models trained on 10×5 instances. SPM-PLAN achieves the lowest makespan in six of the eight settings under greedy strategy and seven of the eight settings under sampling strategy. For inference time, SPM-PLAN (32) is the fastest in six greedy settings, while SPM-PLAN (8) is the fastest in the remaining two settings and all sampling settings. In particular, SPM-PLAN (8) achieves the lowest makespan in four greedy and five sampling settings, while progressively reducing the parameter count by 78.1% (77,314 to 16,922) and the model size from 341kB for SPM-DAN to 228kB for SPM-PLAN (32) and 97kB for SPM-PLAN (8). This observation suggests that PLAN requires only a compact stochastic representation, as its liquid-inspired state updates already capture scheduling-state evolution under processing-time uncertainty. To adapt SPM to

Table 2: Performance comparison on medium- and large-scale deterministic FJSP instances and public benchmarks. Gap (%) denotes the percentage makespan difference from OR-Tools, and schedule generation time is reported. Lower values are better.

Data	Size	OR-Tools Makespan	Greedy						Sampling					
			DANIEL			PLAN			DANIEL			PLAN		
			Makespan	Gap	Time	Makespan	Gap	Time	Makespan	Gap	Time	Makespan	Gap	Time
SD1	30×10	274.67	293.48	6.85	2.44	288.35	4.98	1.91	291.66	6.19	8.69	286.38	4.26	8.09
	40×10	365.96	385.97	5.47	3.25	379.28	3.64	2.52	386.59	5.64	12.60	379.91	3.81	11.89
	100×10	944.20	933.61	-1.12	7.94	920.84	-2.47	6.11	963.33	2.03	52.72	951.21	0.74	59.90
	200×5	1884.70	1893.65	0.47	8.03	1880.33	-0.23	6.55	1991.50	5.67	45.86	1984.71	5.31	53.80
SD2	30×10	692.26	803.74	16.10	2.47	775.64	12.04	1.89	766.45	10.72	8.76	734.24	6.06	8.01
	40×10	998.39	992.15	-0.63	3.24	960.53	-3.79	2.56	963.77	-3.47	12.59	926.98	-7.15	11.89
	100×10	2114.50	2258.44	6.81	7.85	2216.96	4.85	6.20	2303.38	8.93	54.21	2236.00	5.75	61.76
	200×5	5876.30	5945.83	1.18	8.05	5948.38	1.23	6.47	6990.37	18.96	45.76	6802.20	15.76	54.30
Public	Mk	174.20	186.92	7.30	0.80	184.88	6.13	0.58	181.64	4.27	2.27	180.00	3.33	1.68
	rdata	935.80	1029.37	10.00	0.50	1025.34	9.57	0.33	983.60	5.11	1.10	980.42	4.77	0.87
	edata	1028.93	1188.39	15.50	0.45	1176.48	14.34	0.29	1120.95	8.94	1.10	1118.57	8.71	0.83
	vdata	919.60	944.78	2.74	0.45	947.01	2.98	0.28	924.99	0.59	1.11	925.34	0.62	0.81

Table 3: Performance comparison on the stochastic FJSP benchmark SD3. SPM-PLAN is evaluated with hidden dimensions of 32 and 8. The best makespan and inference time within each decoding strategy are highlighted in bold. Lower values are better.

Data	Size	Greedy						Sampling					
		SPM-DAN		SPM-PLAN (32)		SPM-PLAN (8)		SPM-DAN		SPM-PLAN (32)		SPM-PLAN (8)	
		Makespan	Time	Makespan	Time	Makespan	Time	Makespan	Time	Makespan	Time	Makespan	Time
SD3	10×5	718.52	0.85	712.38	0.78	707.96	0.75	675.33	1.56	673.84	1.43	668.68	0.97
	20×5	1319.79	1.84	1255.50	1.78	1263.76	1.70	1275.93	4.34	1226.40	4.34	1238.38	2.95
	15×10	1085.35	2.60	1080.34	2.28	1081.44	2.47	1032.49	8.69	1027.65	8.85	1031.24	5.94
	20×10	1287.89	3.47	1296.59	3.16	1296.30	3.36	1254.90	15.36	1259.91	14.95	1258.02	9.49
	30×10	1884.34	5.20	1858.62	4.67	1839.13	5.00	1870.09	32.56	1853.43	32.13	1819.10	19.92
	40×10	2423.90	7.03	2395.19	6.42	2374.47	6.68	2434.25	55.95	2409.49	56.34	2362.94	35.35
	100×10	5567.87	17.56	5556.26	16.57	5529.57	16.78	5759.71	326.11	5718.01	337.87	5593.38	201.69
	200×5	10489.71	17.26	10734.17	16.02	10667.28	16.63	10881.35	302.72	10905.11	314.58	10700.95	177.09

Table 4: SPM-PLAN performance across hidden dimensions on SD3, with model size in kB. The best makespan within each decoding strategy is highlighted in bold.

Hidden Dim.	32	16	8	4	2
Model Size	228	126	97	89	85
Greedy	611.0	618.2	610.3	620.2	611.8
Sampling	586.8	588.7	585.0	592.6	587.8

PLAN, we tune its hidden dimension on the 10×5 training instances. Table 4 further shows that a hidden dimension of 8 achieves the best overall balance between scheduling performance and efficiency. Figure 2(a) shows that incorporating SPM improves makespan for both DANIEL and PLAN, although it increases inference time. SPM-DAN therefore performs better than DANIEL under stochastic processing times, while SPM-PLAN similarly improves upon PLAN. Nevertheless, DANIEL and SPM-DAN remain inferior to PLAN and SPM-PLAN, respectively, indicating that PLAN provides a stronger scheduling representation both with and without stochastic modelling. Figure 3(a) further compares the normalized performance differences across the SD3 benchmark settings. While PLAN maintains generally lower makespan, its inference-time advantage becomes larger

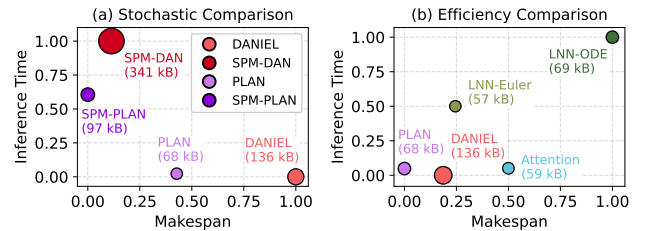


Figure 2: Normalized average performance trade-offs across scheduling models. Lower values are better, and marker size indicates model size. (a) Stochastic FJSP. (b) Ablation study.

as the benchmark index increases, indicating better scalability to larger stochastic FJSP instances.

Multi-faceted Dynamic FJSP

We evaluate PLAN on the multi-faceted dynamic FJSP setting (Liu et al. 2026), which jointly considers dynamic job arrivals, processing-time variation, and machine breakdowns, controlled by p , σ , and μ , respectively. This setting evaluates whether PLAN remains effective under multiple simultaneous scheduling dynamics. Following the original framework, we replace its HGT scheduler with PLAN and retain all other

Table 5: Performance comparison between HGT and PLAN on the multi-faceted dynamic FJSP benchmark under different dynamic conditions. The best makespan and inference time are highlighted in bold. Lower values are better.

Dataset	$p = 0.4, \sigma = 1, \mu = 0.01$				$p = 0.5, \sigma = 1, \mu = 0.01$				$p = 0.4, \sigma = 1, \mu = 0.015$			
	HGT		PLAN		HGT		PLAN		HGT		PLAN	
	Makespan	Time	Makespan	Time	Makespan	Time	Makespan	Time	Makespan	Time	Makespan	Time
10×5	264.20	2.09	259.50	1.98	302.00	2.20	278.70	1.92	278.60	2.13	275.60	1.93
15×5	388.90	2.75	379.70	2.34	382.80	2.67	367.00	2.44	391.60	2.92	405.80	2.52
20×5	548.30	3.67	492.20	3.00	516.50	3.59	518.60	3.14	492.80	3.29	496.20	3.10
20×10	307.00	2.87	303.40	2.67	321.30	3.14	317.80	2.80	338.70	3.26	326.40	2.91
30×10	425.20	4.30	397.10	3.52	416.80	4.49	414.40	3.62	428.90	4.45	422.70	3.52
40×10	554.10	5.80	515.00	4.31	547.30	5.88	525.50	4.42	542.90	5.76	542.50	4.52
80×20	596.50	21.32	579.80	8.68	620.35	20.63	617.20	8.48	528.50	21.31	527.45	9.01
90×30	569.10	35.72	566.80	10.57	658.25	34.44	658.55	11.17	564.35	35.13	558.60	10.70

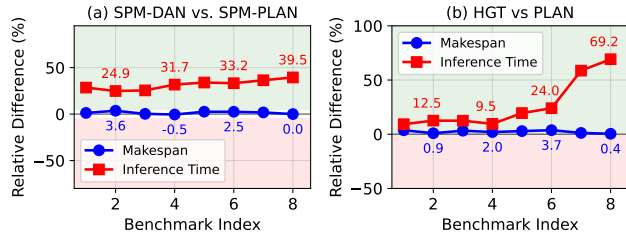


Figure 3: Normalized average performance differences across scheduling models. Benchmark indices follow the corresponding setting order in Tables 3 and 5, from smaller to larger problems. Positive values indicate that PLAN performs better. (a) Stochastic FJSP. (b) Multi-faceted dynamic FJSP.

settings unchanged for a fair comparison, such as the hidden dimension of 128, for which both models are larger than those used in the previous experiments. Both models are trained on 10×5 instances and evaluated on medium- and large-scale settings to assess cross-scale generalization.

Table 5 shows the results under different dynamic conditions. PLAN achieves lower makespan in most settings and lower inference latency than HGT across all reported settings, while reducing the parameter count by 54.8% (1,571,099 to 710,939) and the model size from 6.2MB to 2.8MB. Supplementary Tables S6–S8 evaluate the two models under additional job arrival rates, machine breakdown rates, and processing time variations, further confirming the robustness of PLAN. These results show that PLAN offers a better performance-efficiency trade-off than a deep attention-based HGT learner. Similar to Figure 3(a), Figure 3(b) examines the performance differences across benchmark settings, following the order in Table 5. The makespan improvement remains relatively stable, whereas the time gap widens at larger problem scales.

Ablation Study

We conduct an ablation study to examine the contributions of the liquid-inspired state update and lightweight attention components in PLAN. We compare four representation learning architectures, including LNN-ODE with continuous liq-

uid dynamics, LNN-Euler with parallel liquid approximation, Attention with lightweight attention only, and the full version of PLAN, with DANIEL as the baseline. The variants are evaluated on representative 10×5 and 100×10 deterministic FJSP instances. Figure 2(b) compares their scheduling performance, inference efficiency, and model complexity to illustrate the resulting trade-offs, while Supplementary Table S9 reports the detailed makespan and inference time. The figure shows that replacing continuous ODE dynamics with the Euler approximation substantially reduces inference time and improves scheduling makespan. LNN-Euler already achieves performance comparable to DANIEL with a much smaller model. In contrast, the lightweight Attention-only model is computationally efficient but exhibits weaker representation capability. By combining lightweight attention with the liquid-inspired state update, PLAN achieves the best overall trade-off between scheduling performance, inference efficiency, and model complexity, while reducing the model size significantly from DANIEL’s 136kB to 68kB.

Conclusion

In this paper, we proposed PLAN, a lightweight representation learning framework that combines parallel liquid-inspired state updates with lightweight attention for DRL-based FJSP. PLAN provides a compact alternative to deep attention-based representation learners by modeling scheduling-state evolution while retaining efficient global context aggregation. On deterministic FJSP benchmarks, PLAN achieves better scheduling performance with lower inference latency and a smaller model size than representative baselines. PLAN also maintains its advantage when trained on small instances and tested on unseen larger settings. In stochastic FJSP, PLAN integrates effectively with scenario aggregation, where the compact SPM-PLAN variant reduces the model size from 341kB to 97kB while outperforming SPM-DAN in most settings. In multi-faceted dynamic FJSP, PLAN improves both makespan and inference latency over HGT. The ablation results further confirm that Euler-based liquid approximation and lightweight attention jointly provide the best performance-efficiency trade-off. Overall, PLAN enables faster scheduling decisions and more scheduling trials within a fixed computational budget.

References

- Akpınar, M. H.; Atila, O.; Sengur, A.; Salvi, M.; and Acharya, U. 2025. A novel uncertainty-aware liquid neural network for noise-resilient time series forecasting and classification. *Chaos, Solitons & Fractals*, 193: 116130.
- Brandimarte, P. 1993. Routing and scheduling in a flexible job shop by tabu search. *Ann. Oper. Res.*, 41(1–4): 157–183.
- Burdett, R. L.; and Kozan, E. 2018. An integrated approach for scheduling health care activities in a hospital. *European Journal of Operational Research*, 264(2): 756–773.
- Ghaedy-Heidary, E.; Nejati, E.; Ghasemi, A.; and Torabi, S. A. 2024. A simulation optimization framework to solve Stochastic Flexible Job-Shop Scheduling Problems—Case: Semiconductor manufacturing. *Computers & Operations Research*, 163: 106508.
- Islam, S.; Elmekki, H.; Elsebai, A.; Bentahar, J.; Drawel, N.; Rjoub, G.; and Pedrycz, W. 2024. A comprehensive survey on applications of transformers for deep learning tasks. *Expert Systems with Applications*, 241: 122666.
- Kaleta, M.; and Śliwiński, T. 2026. A neural-driven constructive heuristic for the flexible job shop scheduling problem: An efficient alternative to complex deep learning methods. *Computers & Operations Research*, 191: 107444.
- Kannan, D. D.; Li, W.; Zhang, W.; Wang, J.; Seh, Z. W.; and Ng, M.-F. 2026. When Smaller Wins: Dual-Stage Distillation and Pareto-Guided Compression of Liquid Neural Networks for Edge Battery Prognostics. 28th International Conference on Pattern Recognition, ICPR.
- Kim, M.-S.; Oh, S.-C.; Chang, E. H.; Lee, S.; Wells, J. W.; Arinez, J.; and Jang, Y. J. 2022. A dynamic programming-based heuristic algorithm for a flexible job shop scheduling problem of a matrix system in automotive industry. In *2022 IEEE 18th International Conference on Automation Science and Engineering (CASE)*, 777–782.
- Kwan, Z.; Zhang, W.; Ng, A. B.; Wang, Z.; and See, S. 2026. ReLA: Representation Learning and Aggregation for Job Scheduling with Reinforcement Learning. *arXiv preprint arXiv:2601.03646*.
- Li, X.; and Gao, L. 2016. An effective hybrid genetic algorithm and tabu search for flexible job shop scheduling problem. *International Journal of Production Economics*, 174: 93–110.
- Li, Z.; Qian, B.; Hu, R.; Chang, L.; and Yang, J. 2019. An elitist nondominated sorting hybrid algorithm for multi-objective flexible job-shop scheduling problem with sequence-dependent setups. *Knowledge-Based Systems*, 173: 83–112.
- Liu, G.; Liao, M.; Chen, W.; Zhang, Z.; Wan, H.; and Lin, Y. 2026. Multi-faceted dynamic flexible job shop scheduling via heterogeneous graph transformer and deep reinforcement learning. *Expert Systems with Applications*, 303: 130532.
- Liu, X.; Chen, X.; Chau, V.; Musial, J.; and Blazewicz, J. 2025. Flexible Job Shop Scheduling Problem using graph neural networks and reinforcement learning. *Computers & Operations Research*, 182: 107139.
- Sels, V.; Gheysen, N.; and Vanhoucke, M. 2012. A comparison of priority rules for the job shop scheduling problem under different flow time- and tardiness-related objective functions. *International Journal of Production Research*, 50(15): 4255–4270.
- Smit, I. G.; Wu, Y.; Troubil, P.; Zhang, Y.; and Nuijten, W. P. 2025. Neural Combinatorial Optimization for Stochastic Flexible Job Shop Scheduling Problems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39(25), 26678–26687.
- Song, W.; Chen, X.; Li, Q.; and Cao, Z. 2023. Flexible Job-Shop Scheduling via Graph Neural Network and Deep Reinforcement Learning. *IEEE Transactions on Industrial Informatics*, 19(2): 1600–1610.
- Wang, R.; Wang, G.; Sun, J.; Deng, F.; and Chen, J. 2024. Flexible Job Shop Scheduling via Dual Attention Network-Based Reinforcement Learning. *IEEE Transactions on Neural Networks and Learning Systems*, 35(3): 3091–3102.
- Wang, X.; Yu, X.; Wang, Z.; Si, Z.; Wu, G.; Su, X.-c.; Qu, S.; Xiong, B.; Peng, H.; Li, X.; and Wang, L. 2026. Machine Learning-Driven Combinatorial Optimization: A Systematic Review. *Archives of Computational Methods in Engineering*.
- Xie, J.; Gao, L.; Peng, K.; Li, X.; and Li, H. 2019. Review on flexible job shop scheduling. *IET Collaborative Intelligent Manufacturing*, 1(3): 67–77.

Supplementary material

This supplementary material provides additional experimental settings, results, and details.

Deterministic FJSP

Supplementary Table 6 provides the complete hyperparameter settings for training via PPO, PLAN, and generating the dataset for deterministic FJSP.

Table 6: Training, PLAN architecture, and dataset generation settings used for deterministic FJSP experiments.

Training (PPO)	PLAN Architecture	Deterministic Dataset
Optimizer: Adam	Operation feature dim: 10	Processing time: [1, 99]
Learning rate: 3×10^{-4}	Machine feature dim: 8	Compatible machines/operation: 1 - 5
Discount factor (γ): 1.0	Hidden dimensions: [32, 8]	Operations/job: Equal to # machines
PPO epochs: 4	Attention heads (Operation): [4,4]	Training instances: 100
PPO clip (ϵ): 0.2	Attention heads (Machine): [4,4]	Training size: 10×5 - 20×10
GAE (λ): 0.98	Actor hidden dim: 64	Test instances: 100
Mini-batch size: 1024	Critic hidden dim: 64	Evaluation: Greedy and Sampling (100)
Training environments: 20	Actor/Critic MLP layers: 3	Generalization: Models trained on 10×5
Maximum updates: 1000	Dropout: 0	Test sizes: 10×5 - 200×5
Validation interval: 10	Model size: 68 kB	
Training/Test seed: 0-4		

In Supplementary Table 7, we compare our proposed PLAN-based scheduling with existing solutions like OR-Tools, top Priority dispatching rules (SPT, MWKR), and DRL-based methods (HGNN, DANIEL). The results show that PLAN consistently achieves superior scheduling quality, outperforming both conventional optimization and state-of-the-art learning-based baselines.

Table 7: Comparison with OR-Tools, PDRs, and DRL-based methods on benchmark instances.

Data	Size	OR-Tools	PDRs		Greedy (DRL)			Sampling (DRL)		
			SPT	MWKR	HGNN	DANIEL	PLAN	HGNN	DANIEL	PLAN
SD1	10×5	96.32	129.82	113.23	111.67	107.97	107.36	105.59	102.36	101.48
	20×5	188.15	230.48	209.78	211.22	197.66	197.78	207.53	193.83	193.14
	15×10	143.53	198.33	171.25	166.92	160.78	159.21	160.86	152.51	150.68
	20×10	195.98	255.17	216.11	215.78	199.14	198.21	214.81	195.30	193.38
	30×10	274.67	350.07	312.93	314.71	293.48	288.35	308.55	291.66	286.38
	40×10	365.96	445.17	414.82	417.87	385.97	379.28	410.76	386.59	379.91
SD2	10×5	326.24	514.39	549.28	553.61	413.91	407.86	483.90	366.59	362.46
	20×5	602.04	835.94	1026.03	1059.04	673.28	661.45	962.90	632.85	624.68
	15×10	377.17	703.07	830.53	807.47	588.14	587.84	756.07	519.78	515.61
	20×10	464.16	829.14	1040.69	1045.82	606.14	603.43	990.37	552.09	550.11
	30×10	692.26	1105.99	1539.67	1564.57	803.74	775.64	1486.56	766.45	734.24
	40×10	998.39	1357.16	2037.65	2048.96	992.15	960.53	1976.25	963.77	926.98

Stochastic FSJP

Supplementary Table 8 provides the complete hyperparameter settings for training via PPO, SPM-PLAN, and generating the dataset for stochastic FJSP.

Table 8: Training, SPM-PLAN architecture, and stochastic dataset settings used for SFJSP experiments.

Training (PPO)	SPM-PLAN Architecture	Stochastic Dataset
Optimizer: Adam	Operation feature dim: 10	Processing time: [1, 99]
Learning rate: 3×10^{-4}	Machine feature dim: 8	Compatible machines/operation: 1 - 5
Discount factor (γ): 1.0	Hidden dimensions: [32, 8]	Operations/job: Equal to # machines
PPO epochs: 4	Attention heads (Operation): [4,4]	Training instances: 100
PPO clip (ϵ): 0.2	Attention heads (Machine): [4,4]	Training size: 10×5 - 20×10
GAE (λ): 0.98	SAA Attention: Enabled	Test instances: 100
Mini-batch size: 143	SAA attention dim: 8	Input realizations: 100
Gradient accumulation: 7	Scenario aggregation: Mean	Evaluation realizations: 1000
Training environments: 20	Actor/Critic hidden dim: 64	Variance: Random (Lognormal)
Maximum updates: 1000	Actor/Critic MLP layers: 3	Training method: SAA
Validation interval: 10	Dropout: 0	Objective: VaR ($\alpha = 0.95$)
Training seed: 400		Generalization: Models trained on 10×5
Test seed: 50		Test sizes: 10×5 - 200×5

Supplementary Fig. 4 shows the training behaviour of PLAN and DANIEL with and without SPM modules under the stochastic scenario. PLAN without SPM achieves significantly better reward and attains the best-record makespan throughout training than DAN. With SPM, SPM-PLAN converges to a higher final reward than SPM-DAN, demonstrating its effectiveness in stochastic aggregation settings.



Figure 4: Stochastic FJSP (SD3) training behavior

Table 9 shows the makespan and inference time comparison of PLAN and DANIEL with and without SPM, across small, medium, and large instances. Without SPM, PLAN shows consistently higher scheduling makespan than DAN. SPM improves scheduling quality of the FJSP with stochastic processing times, with SPM-PLAN achieving performance gains and reducing the inference time by more than 50% compared to SPM-DAN, offering an efficient solution.

Table 9: Performance $Var_{\alpha}(C_{\max})$ and Efficiency comparison on stochastic FJSP

Data	Decoding	Size	DAN		SPM-DAN		PLAN		SPM-PLAN (32)		SPM-PLAN (8)	
			Makespan	Time	Makespan	Time	Makespan	Time	Makespan	Time	Makespan	Time
SD3	Greedy	10×5	733.92	1.07	718.52	0.85	740.96	0.88	712.38	0.78	707.96	0.75
		20×5	1341.80	2.05	1319.79	1.84	1297.29	1.66	1255.50	1.78	1263.76	1.70
		15×10	1127.60	3.19	1085.35	2.60	1114.99	2.42	1080.34	2.28	1081.44	2.47
		20×10	1376.29	4.24	1287.89	3.47	1362.11	3.57	1296.59	3.16	1296.30	3.36
		30×10	1971.57	6.18	1884.34	5.20	1970.33	5.20	1858.62	4.67	1839.13	5.00
		40×10	2541.06	8.51	2423.90	7.03	2538.39	6.93	2395.19	6.42	2374.47	6.68
		100×10	6019.35	20.93	5567.87	17.56	5795.76	17.89	5556.26	16.57	5529.57	16.78
		200×5	11531.42	21.18	10489.71	17.26	10633.82	16.95	10734.17	16.02	10667.28	16.63
	Sampling	10×5	684.56	1.24	675.33	1.56	685.87	1.12	673.84	1.43	668.68	0.97
		20×5	1291.56	2.45	1275.93	4.34	1262.44	2.32	1226.40	4.34	1238.38	2.95
		15×10	1076.24	4.38	1032.49	8.69	1065.77	4.01	1027.65	8.85	1031.24	5.94
		20×10	1340.44	6.60	1254.90	15.36	1324.40	6.23	1259.91	14.95	1258.02	9.49
		30×10	1933.25	11.39	1870.09	32.56	1943.70	11.38	1853.43	32.13	1819.10	19.92
		40×10	2522.82	18.42	2434.25	55.95	2530.06	18.01	2409.49	56.34	2362.94	35.35
		100×10	6088.92	85.62	5759.71	326.11	5886.58	94.76	5718.01	337.87	5593.38	201.69
		200×5	11547.33	64.40	10881.35	302.72	11102.79	70.76	10905.11	314.58	10700.95	177.09
Model Size (kB)			136		341		68		228		97	

Multi-faceted Dynamic FJSP

Supplementary Table 10 provides the complete hyperparameter settings for training, PLAN, and generating the dataset for Multi-faceted Dynamic FJSP.

Table 10: Training, PLAN architecture, and MFDFJSP dataset settings.

Training PPO	PLAN Architecture	Multi-faceted Dynamic Dataset
Optimizer = AdamW	Embedding dimension = 128	Dynamic job arrival (p): 0.3, 0.4, 0.5
Learning rate = 5×10^{-5}	Hidden dimension = 256	Processing time variability (σ): 0.5, 1, 2
Weight decay = 10^{-2}	Encoder layers = 4	Machine failure rate (μ): 0.0005, 0.001, 0.0015
Batch size = 64	Attention heads = 8	Training size = 10×5
Training iterations = 100	Key dimension = 16	Test sizes = 10×5 – 90×30
Epochs / iteration = 3	Dropout = 0.2	Maximum operations = 400
Gradient clipping = 1.0	Operation feature dim = 7	Number of cases = 10
PPO clip = 0.15	Machine feature dim = 4	Generalization: trained on 10×5
Discount factors = 0.925	Arc feature dim = 2	Framework: HGAN replaced by PLAN
Entropy coefficient = 0.01	Representation learner = PLAN	
Policy / Value loss = 1.0 / 1.0		

For Multi-faceted dynamic FJSP, we consider different dynamic settings by varying these parameters: dynamic job arrivals (p), variable processing times (σ), and machine breakdowns (μ). Table 11 presents the results under different dynamic degrees of 0.3, 0.4, and 0.5 with $\sigma = 1$, $\mu = 0.01$. Table 12 shows the results under different rates of machine failure (0.005, 0.01, and 0.015) with $p = 0.4$, $\sigma = 1$. Table 13 shows the results under different processing time probabilities, $\sigma = 0.5, 1, 2$ with $p = 0.4$, $\mu = 0.01$. Across most settings, PLAN consistently outperforms HGT, achieving lower makespan and faster inference

Table 11: Performance comparison under different dynamic degrees.

Dataset	$p = 0.3, \sigma = 1, \mu = 0.01$				$p = 0.4, \sigma = 1, \mu = 0.01$				$p = 0.5, \sigma = 1, \mu = 0.01$			
	HGT		PLAN		HGT		PLAN		HGT		PLAN	
	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)
10×5	266.00	2.16	264.40	1.88	264.20	2.09	259.50	1.98	302.00	2.20	278.70	1.92
15×5	405.60	2.81	368.10	2.41	388.90	2.75	379.70	2.34	382.80	2.67	367.00	2.44
20×5	509.10	3.35	471.00	2.93	548.30	3.67	492.20	3.00	516.50	3.59	518.60	3.14
20×10	314.00	2.93	301.60	2.63	307.00	2.87	303.40	2.67	321.30	3.14	317.80	2.80
30×10	392.60	4.10	386.10	3.30	425.20	4.30	397.10	3.52	416.80	4.49	414.40	3.62
40×10	524.80	5.57	506.60	4.25	554.10	5.80	515.00	4.31	547.30	5.88	525.50	4.42

Table 12: Performance comparison under different machine failure rates.

Dataset	$p = 0.4, \sigma = 1, \mu = 0.005$				$p = 0.4, \sigma = 1, \mu = 0.01$				$p = 0.4, \sigma = 1, \mu = 0.015$			
	HGT		PLAN		HGT		PLAN		HGT		PLAN	
	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)
10×5	259.70	2.02	257.50	1.82	264.20	2.09	259.50	1.98	278.60	2.13	275.60	1.93
15×5	384.60	2.62	372.40	2.48	388.90	2.75	379.70	2.34	391.60	2.92	405.80	2.52
20×5	484.50	3.26	474.30	2.98	548.30	3.67	492.20	3.00	492.80	3.29	496.20	3.10
20×10	296.80	2.90	288.30	2.61	307.00	2.87	303.40	2.67	338.70	3.26	326.40	2.91
30×10	403.70	4.11	424.20	3.49	425.20	4.30	397.10	3.52	428.90	4.45	422.70	3.52
40×10	524.30	5.62	532.60	4.48	554.10	5.80	515.00	4.31	542.90	5.76	542.50	4.52

Table 13: Performance comparison under different probabilities of processing time.

Dataset	$p = 0.4, \sigma = 0.5, \mu = 0.01$				$p = 0.4, \sigma = 1, \mu = 0.01$				$p = 0.4, \sigma = 2, \mu = 0.01$			
	HGT		PLAN		HGT		PLAN		HGT		PLAN	
	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)	Makespan	Time (s)
10×5	277.80	2.02	253.20	1.75	264.20	2.09	259.50	1.98	293.50	2.23	299.30	1.98
15×5	397.50	2.73	372.40	2.41	388.90	2.75	379.70	2.34	404.60	2.83	402.30	2.47
20×5	545.60	3.54	494.10	3.04	548.30	3.67	492.20	3.00	470.20	3.34	475.60	2.90
20×10	296.30	3.08	295.30	2.64	307.00	2.87	303.40	2.67	326.80	2.99	305.30	2.63
30×10	427.10	4.30	390.90	3.36	425.20	4.30	397.10	3.52	420.30	4.38	426.60	3.52
40×10	559.60	5.72	520.20	4.39	554.10	5.80	515.00	4.31	510.30	5.44	527.90	4.52

Detailed Ablation results

Supplementary Fig. 5 shows the training behaviour of LNN with ODE, discretized LNN, Simple Attention, and PLAN compared with DANIEL. The original continuous ODE-based LNN fails to adapt to the discrete FSJP dynamics, whereas discretization with Euler yields performance competitive with DANIEL. By combining the liquid-inspired state update with simple attention, PLAN reaches a higher final training reward.

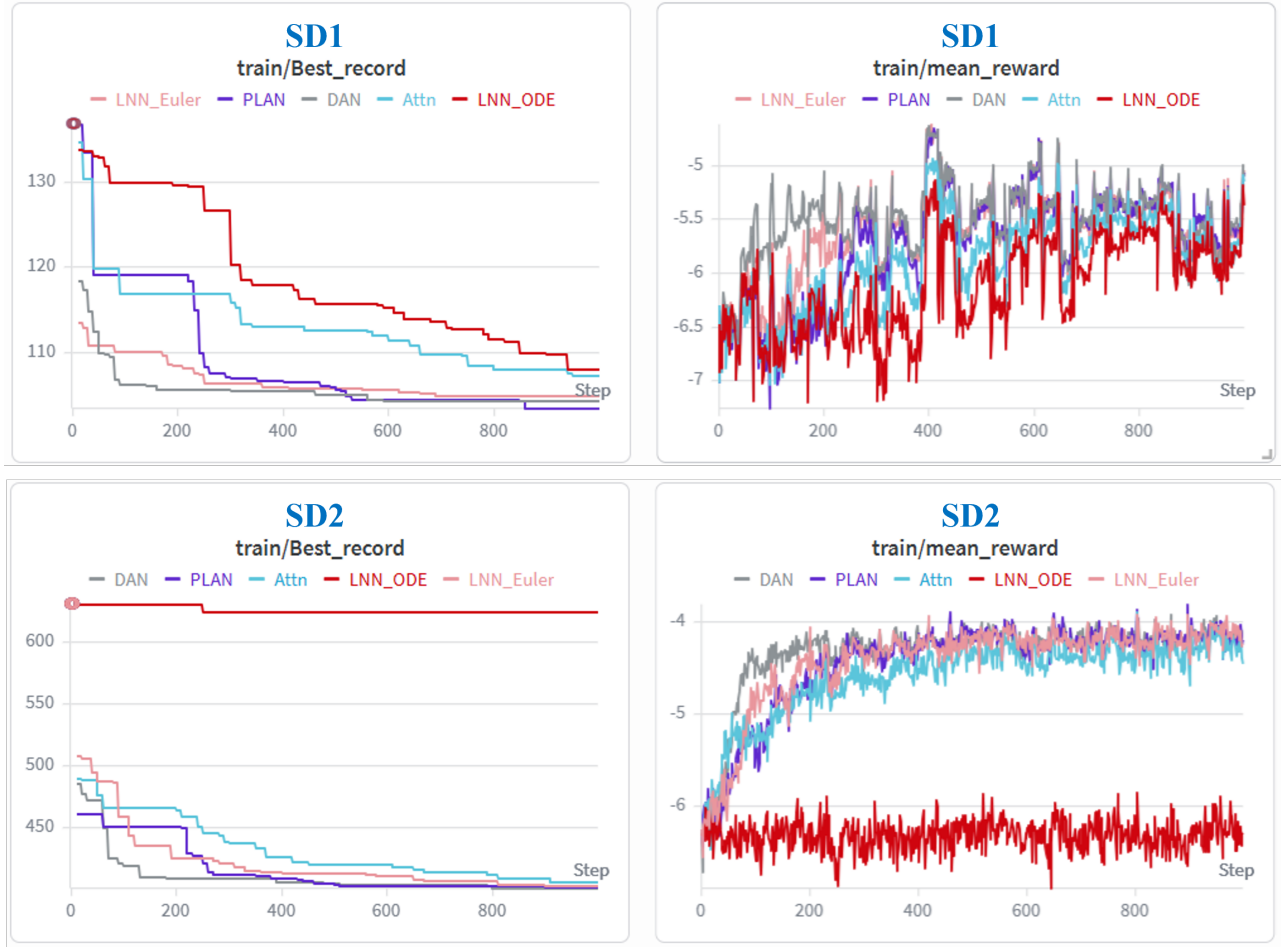


Figure 5: Ablation Training behavior

Supplementary Table 14 shows the makespan and inference time across 10×5 and 100×10 instance sizes in greedy and sampling encoding. To evaluate generalization, the 100×10 instances are tested using the model trained on the 10×5 dataset. The results show that PLAN offers a better performance-efficiency trade-off than DANIEL in both small and large problem scales.

Table 14: Ablation study on 10x5 and 100x10 dataset

Data	Size	Metric	Greedy					Sampling				
			DANIEL	LNN_ODE	LNN_Euler	Attention	PLAN	DANIEL	LNN_ODE	LNN_Euler	Attention	PLAN
SD1	10×5	Makespan	108.30	110.53	108.36	110.06	107.22	102.37	103.43	102.19	103.57	101.67
		Inf Time (s)	0.48	6.70	0.49	0.31	0.33	1.15	7.90	1.19	0.82	0.81
	100×10	Makespan	933.61	1066.36	989.55	949.65	920.84	963.33	1050.78	1017.18	1007.15	951.21
		Inf Time (s)	7.94	154.11	43.38	6.04	6.11	52.72	435.37	92.66	60.48	59.90
SD2	10×5	Makespan	409.75	502.27	407.78	415.00	407.86	363.96	421.74	362.32	362.12	362.46
		Inf Time (s)	0.40	6.06	0.61	0.44	0.31	1.11	7.24	1.16	0.85	0.81
	100×10	Makespan	2258.44	Invalid	2232.14	2275.00	2216.96	2303.38	Invalid	2267.55	2459.75	2236.00
		Inf Time (s)	7.94	–	46.80	5.96	6.20	54.21	–	95.99	61.80	61.76
Model Size (kB)			135	69	57	59	68	135	69	57	59	68